Taller Práctico de Repaso y Consolidación en C++: Recursividad, Búsqueda Secuencial y Punteros con Método 5R

Aramis Demaria

E.E.S.T. N.º 1 “Eduardo Ader” – Vicente López

Laboratorio de Programación (LPR) – 5.º Año 3.º A-B — Grupo A

Prof. Mansilla Muñoz York Elías

2026 — Grupo A

Resumen

Se consolidan tres pilares en una suite unificada: recursividad con caso base, búsqueda secuencial O(n) en arreglo contiguo con *break* y verificación de contigüidad, e intercambio con punteros y variable temporal. La metodología 5R guía el aprendizaje y la documentación. Las pruebas con suma 1 a 5, búsqueda de 18 en índice 6 e intercambio 100/500 validan la implementación. Se incluyen las cinco preguntas del cuestionario.

Palabras clave: recursividad, caso base, desbordamiento de pila, búsqueda secuencial, punteros, método 5R

Abstract

Three pillars are consolidated: recursion with base case, O(n) sequential search with break, and pointer swap. The 5R methodology guides learning and a five-item questionnaire is included.

Keywords: recursion, base case, sequential search, pointers, 5R method

Introducción

La primera mitad del ciclo abordó recursividad, arreglos y punteros por separado. La Actividad 6 los integra en un solo ejecutable para exigir el cambio de modelo mental entre pila, arreglo y punteros.

Revisión de la Literatura

La recursividad requiere caso base para cortar la cadena; sin él hay recursión infinita. La pila es región LIFO para marcos de llamada. La búsqueda secuencial recorre índice por índice y *break* evita iteraciones tras el hallazgo. El arreglo es contiguo con incremento de *sizeof(int)*. El intercambio con punteros requiere variable temporal para no perder dato (Cormen et al., 2009).

Reto	Núcleo (20%)	Riesgo si se omite
Recursividad	if (n <= 0) return 0	Desbordamiento de pila
Búsqueda	break tras hallar	Recorrido O(n) innecesario
Intercambio	int temp = *pA	Pérdida de dato

Nota. Núcleos críticos.

Método

La suite en *src/main.cpp* declara *sumaRecursiva* e *intercambiarValores*. La compilación se ejecutó con *g++ src/main.cpp -o src/repasogeneral.exe*. El versionado es *v1.0.0 Baseline* y el doble remoto permite envío a GitHub y GitLab.

Resultados

La suma de 1 a 5 resultó 15. La búsqueda halló 18 en índice 6 con dirección hexadecimal e interrumpió con *break*. El intercambio invirtió 100 y 500. La contigüidad se verificó imprimiendo direcciones con incremento de 4 bytes.

Cuestionario Técnico de Control

Se responden las cinco preguntas de la guía de la Actividad 6.

Pregunta 1 — Caso Base

Enunciado: ¿Qué error crítico ocurre si una recursión carece de caso base?

Respuesta: Recursión infinita que apila marcos hasta agotar la pila —desbordamiento de pila (*stack overflow*)— y provoca terminación abrupta del programa. El caso base *if (n <= 0) return 0* corta la cadena apilando y desapilando correctamente los retornos.

Pregunta 2 — Eficiencia con break

Enunciado: ¿Por qué break optimiza el rendimiento en búsqueda secuencial?

Respuesta: Porque al hallar el elemento interrumpe el ciclo *for* y evita iterar el resto del arreglo. En notación O(n), pasar de *n* a *k* comparaciones —donde *k* es el índice

hallado— ahorra ciclos de procesador, relevante en arreglos de miles de sensores del Proyecto Integrador Anual.

Pregunta 3 — Direccionamiento y Contigüidad

Enunciado: ¿Cómo comprobar con & que los elementos son contiguos?

Respuesta: Imprimiendo *&vectorDatos[i]* en ciclo: las direcciones crecen de forma constante en *sizeof(int)* —4 bytes— sin huecos, evidenciando almacenamiento contiguo. Esta propiedad permite aritmética de punteros y acceso por índice con desplazamiento lineal.

Pregunta 4 — Variables Temporales

Enunciado: ¿Qué función cumple la variable temporal en el intercambio con punteros?

Respuesta: Resguarda el valor original de **pA* antes de sobrescribirlo con **pB*. Sin *temp*, la secuencia **pA = *pB; *pB = *pA* perdería el valor de **pA* y ambos quedarían con el valor de **pB*. La variable temporal preserva el dato durante la permutación física en memoria.

Pregunta 5 — Autoevaluación

Enunciado: ¿Cuál reto resultó más complejo y cómo lo resolviste?

Respuesta: Respuesta personal del estudiante: la recursividad resultó más compleja por visualizar la pila. Se resolvió trazando manualmente las llamadas *5→4→3→2→1→0* y observando cómo se apilan los marcos y se desapilan los retornos, complementado con depuración paso a paso y con el esquema de memoria de la guía.

Discusión

La variable temporal es indispensable, el caso base es el fusible contra desbordamiento y *break* optimiza el uso de procesador. La evidencia de contigüidad valida el modelo de memoria contigua.